

MONGODB BASIC PRACTICE TASK

MongoDB CRUD Operations, Query Operators and E-Commerce Product Listing

Name: Elamathi R

Course: Full Stack Web Development

Technology: MongoDB

OBJECTIVE

To understand the basics of MongoDB, its commands, and apply the knowledge by performing CRUD operations, practicing query operators, and implementing a small real-world E-Commerce product listing use case.

1. DATABASE & COLLECTION SETUP

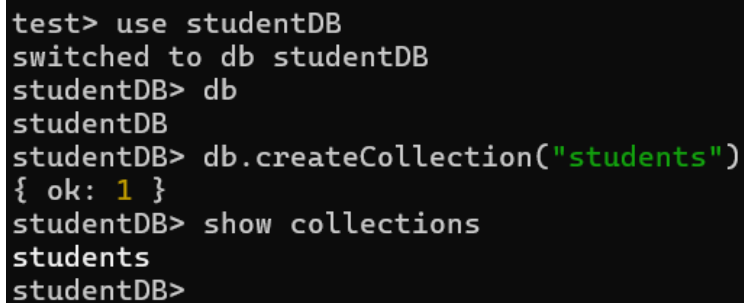
Description:

A new MongoDB database named studentDB was created, along with a students collection for storing student information.

MongoDB Commands:

```
use studentDB
db.createCollection("students")
show collections
```

Execution Screenshot:



```
test> use studentDB
switched to db studentDB
studentDB> db
studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> show collections
students
studentDB>
```

2. INSERT OPERATIONS

Description:

Documents were inserted into the students collection using insertOne() for a single document and insertMany() for multiple documents.

MongoDB Commands:

```
db.students.insertOne({
  name: "Arun",
  age: 21,
  course: "MERN Stack",
  status: "enrolled"
})
```

```
db.students.insertMany([
  {
    name: "Priya",
    age: 22,
    course: "MERN Stack",
    status: "enrolled"
  },
  {
    name: "Mathi",
    age: 23,
    course: "Java",
    status: "completed"
  },
  {
    name: "Divya",
    age: 20,
    course: "Python",
    status: "enrolled"
  }
])
```

Execution Screenshots:

```
studentDB> db.students.insertOne({
...   name: "Arun",
...   age: 21,
...   course: "MERN Stack",
...   status: "enrolled"
... })
{
  acknowledged: true,
  insertedId: ObjectId('6ac19e8b0efb7fa4cb1e2621')
}
studentDB>
```

```
studentDB> db.students.insertMany([
...   {
...     name: "Priya",
...     age: 22,
...     course: "MERN Stack",
...     status: "enrolled"
...   },
...   {
...     name: "Mathi",
...     age: 23,
...     course: "Java",
...     status: "completed"
...   },
...   {
...     name: "Divya",
...     age: 20,
...     course: "Python",
...     status: "enrolled"
...   }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    '1': ObjectId('6ac1a0f70efb7fa4cb1e2623'),
    '2': ObjectId('6ac1a0f70efb7fa4cb1e2624')
  }
}
studentDB>
```

3. READ OPERATIONS

Description:

The find() operation was used to retrieve documents from the students collection. First, all student documents were fetched. Then, a filter was used to retrieve only students enrolled in the MERN Stack course.

MongoDB Command – Fetch All Documents:

```
db.students.find()
```

Execution Screenshot:

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('6ac19e8b0efb7fa4cb1e2621'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2623'),
    name: 'Mathi',
    age: 23,
    course: 'Java',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2624'),
    name: 'Divya',
    age: 20,
    course: 'Python',
    status: 'enrolled'
  }
]
studentDB> |
```

MongoDB Command – Filter by Course:

```
db.students.find({ course: "MERN Stack" })
```

Description:

A filter was used to retrieve only the students enrolled in the MERN Stack course.

Execution Screenshot:

```
studentDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('6ac19e8b0efb7fa4cb1e2621'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
studentDB> |
```

4. UPDATE OPERATIONS

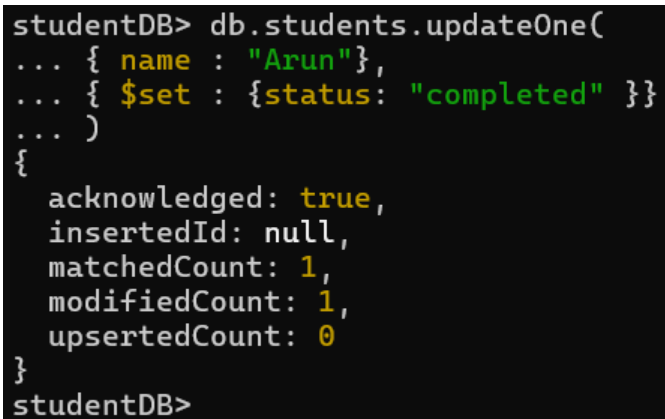
Description:

The updateOne() operation was used to update a single student's status. The updateMany() operation was then used to update the status of multiple students at the same time.

MongoDB Command – Update One Document:

```
db.students.updateOne(
  { name: "Arun" },
  { $set: { status: "completed" } }
)
```

Execution Screenshot:



```
studentDB> db.students.updateOne(
... { name : "Arun"},
... { $set : {status: "completed" }}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB>
```

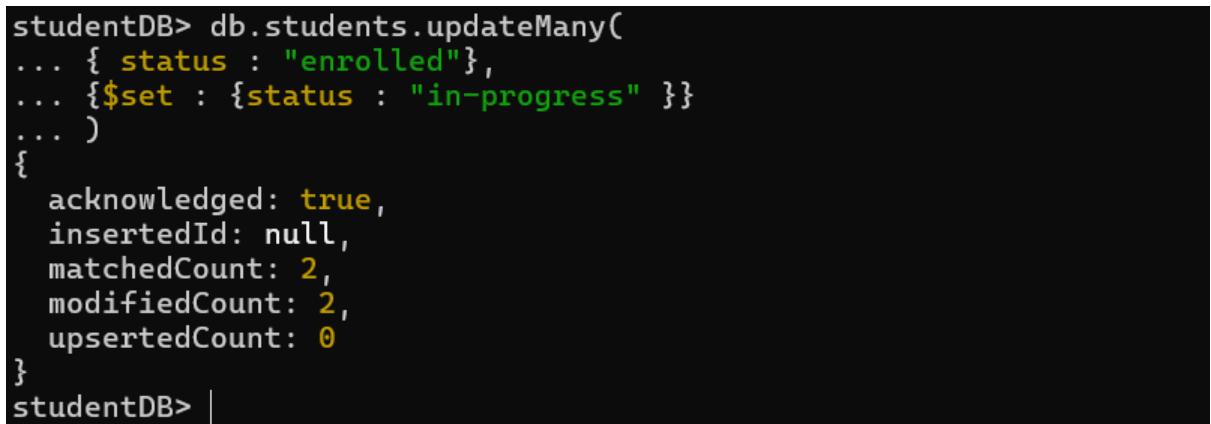
MongoDB Command – Update Multiple Documents:

```
db.students.updateMany(  
  { status: "enrolled" },  
  { $set: { status: "in-progress" } }  
)
```

Description:

The updateMany() operation was used to update the status of all students whose current status was enrolled. Their status was changed to in-progress.

Execution Screenshot:



```
studentDB> db.students.updateMany(  
... { status : "enrolled"},  
... {$set : {status : "in-progress" }}  
... )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 2,  
  modifiedCount: 2,  
  upsertedCount: 0  
}  
studentDB> |
```


5. DELETE OPERATIONS

Description:

Delete operations are used to remove documents from a MongoDB collection. A single document can be deleted based on a specific condition.

MongoDB Command – Delete One Document:

```
db.students.deleteOne({ name: "Divya" })
```

Execution Screenshot:

```
studentDB> db.students.deleteOne({ name: "Divya" })
{ acknowledged: true, deletedCount: 1 }
studentDB>
```

MongoDB Command – Delete All Documents:

```
db.students.deleteMany({})
```

Description:

The deleteMany() operation was used to delete all documents from the students collection for practice purposes.

Execution Screenshot:

```
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
studentDB>
```

6. QUERY OPERATORS

Description:

MongoDB query operators are used to perform conditional searches and retrieve specific documents based on different criteria.

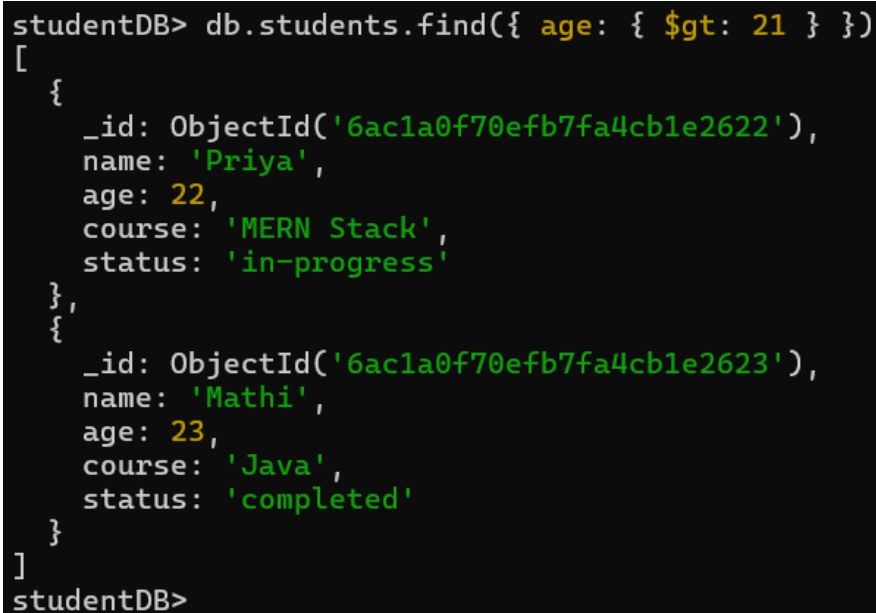
MongoDB Command – \$gt Operator:

```
db.students.find({ age: { $gt: 21 } })
```

Description:

The \$gt operator was used to find students whose age is greater than 21.

Execution Screenshot:



```
studentDB> db.students.find({ age: { $gt: 21 } })
[
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'in-progress'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2623'),
    name: 'Mathi',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
studentDB>
```

MongoDB Command – \$lt Operator:

```
db.students.find({ age: { $lt: 23 } })
```

Description:

The \$lt operator was used to find students whose age is less than 23.

Execution Screenshot:

```
studentDB> db.students.find({ age: {$lt:23 } })
[
  {
    _id: ObjectId('6ac19e8b0efb7fa4cb1e2621'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'in-progress'
  }
]
studentDB>
```

MongoDB Command – \$in Operator:

```
db.students.find({
  course: { $in: ["MERN Stack", "Java"]}
})
```

Description:

The \$in operator was used to find students whose course is either MERN Stack or Java.

Execution Screenshot:

```
studentDB> db.students.find({
...   course: {$in: ["MERN Stack", "Java"]}
... })
[
  {
    _id: ObjectId('6ac19e8b0efb7fa4cb1e2621'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'in-progress'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2623'),
    name: 'Mathi',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
studentDB>
```

MongoDB Command – \$and Operator:

```
db.students.find({
  $and: [
    { age: { $gt: 20 } },
    { course: "MERN Stack" }
  ]
})
```

Description:

The \$and operator was used to find students whose age is greater than 20 and whose course is MERN Stack.

Execution Screenshot:

```
studentDB> db.students.find({
... $and:[
... {age: {$gt:20} },
... {course: "MERN Stack" }
... ]
... })
[
  {
    _id: ObjectId('6ac19e8b0efb7fa4cb1e2621'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'in-progress'
  }
]
studentDB>
```

MongoDB Command – \$or Operator:

```
db.students.find({
  $or: [
    { course: "MERN Stack" },
    { course: "Java" }
  ]
})
```

Description:

The \$or operator was used to find students whose course is either MERN Stack or Java.

Execution Screenshot:

```
studentDB> db.students.find({
... $or:[
... {course: "MERN Stack"},
... {course: "Java" }
... ]
... })
[
  {
    _id: ObjectId('6ac19e8b0efb7fa4cb1e2621'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2622'),
    name: 'Priya',
    age: 22,
    course: 'MERN Stack',
    status: 'in-progress'
  },
  {
    _id: ObjectId('6ac1a0f70efb7fa4cb1e2623'),
    name: 'Mathi',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
studentDB> |
```

MongoDB Command – \$exists Operator:

```
db.students.find({
  phone: { $exists: true }
})
```

Description:

The \$exists operator was used to find students where the phone field exists in the document.

Execution Screenshot:

```
studentDB> db.students.find({
... phone: { $exists: true }
... })
[
  {
    _id: ObjectId('6ac19e8b0efb7fa4cb1e2621'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed',
    phone: '9876543210'
  }
]
studentDB> |
```

7. REAL-WORLD USE CASE – E-COMMERCE PRODUCT LISTING

Description:

An E-Commerce Product Listing system was implemented as a real-world use case. A products collection was created to store product details such as name, price, category, and stock. Insert, search, and update operations were performed on the product data.

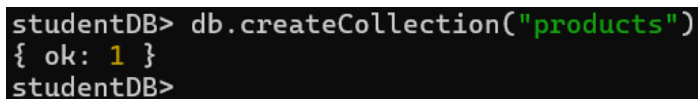
MongoDB Command – Create Products Collection:

```
db.createCollection("products")
```

Description:

A products collection was created to store the details of products in the E-Commerce system.

Execution Screenshot:

A screenshot of a terminal window showing the execution of a MongoDB command. The prompt is 'studentDB>'. The command entered is 'db.createCollection("products")'. The output is '{ ok: 1 }'. The prompt 'studentDB>' is shown again on the next line.

```
studentDB> db.createCollection("products")
{ ok: 1 }
studentDB>
```

MongoDB Command – Insert One Product:

```
db.products.insertOne({
  name: "Laptop",
  price: 55000,
  category: "Electronics",
  stock: 10
})
```

Description:

A single product document was inserted into the products collection with details such as product name, price, category, and available stock.

Execution Screenshot:

```
studentDB> db.products.insertOne({
... name: "Laptop",
... price: 55000,
... category: "Electronics",
... stock: 10
... })
{
  acknowledged: true,
  insertedId: ObjectId('6ac1ae9b0efb7fa4cb1e2625')
}
studentDB>
```

MongoDB Command – Insert Multiple Products:

```
db.products.insertMany([
{
  name: "Smartphone",
  price: 25000,
  category: "Electronics",
  stock: 20
},
{
  name: "Keyboard",
  price: 1500,
  category: "Accessories",
  stock: 30
},
{
  name: "Headphones",
  price: 3000,
  category: "Accessories",
  stock: 15
}
])
```

Description:

Multiple product documents were inserted into the products collection to demonstrate how an E-Commerce system can store several products at once.

Execution Screenshot:

```
studentDB> db.products.insertMany([
... {
...   name: "Smartphone",
...   price: 25000,
...   category: "Electronics",
...   stock: 20
... },
... {
...   name: "Keyboard",
...   price: 1500,
...   category: "Accessories",
...   stock: 30
... },
... {
...   name: "Headphones",
...   price: 3000,
...   category: "Accessories",
...   stock: 15
... }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac1afed0efb7fa4cb1e2626'),
    '1': ObjectId('6ac1afed0efb7fa4cb1e2627'),
    '2': ObjectId('6ac1afed0efb7fa4cb1e2628')
  }
}
studentDB>
```

MongoDB Command – Search Products by Category:

```
db.products.find({
  category: "Electronics"
})
```

Description:

The find() operation was used to search for products belonging to the Electronics category.

Execution Screenshot:

```
studentDB> db.products.find({
... category: "Electronics"
... })
[
  {
    _id: ObjectId('6ac1ae9b0efb7fa4cb1e2625'),
    name: 'Laptop',
    price: 55000,
    category: 'Electronics',
    stock: 10
  },
  {
    _id: ObjectId('6ac1afed0efb7fa4cb1e2626'),
    name: 'Smartphone',
    price: 25000,
    category: 'Electronics',
    stock: 20
  }
]
studentDB> |
```

MongoDB Command – Update Product Stock:

```
db.products.updateOne(
  { name: "Laptop" },
  { $set: { stock: 5 } }
)
```

Description:

The updateOne() operation was used to update the stock quantity of the Laptop product from 10 to 5.

Execution Screenshot:

```
studentDB> db.products.updateOne(
... {name: "Laptop"},
... {$set: { stock : 5 }}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB>
```

8.CONCLUSION

Through this practice task, I learned the basics of MongoDB and performed CRUD operations using MongoDB commands. I practiced inserting, reading, updating, and deleting documents, along with using query operators such as \$gt, \$lt, \$in, \$and, \$or, and \$exists. I also applied these concepts to a real-world E-Commerce Product Listing use case by creating products, inserting product data, searching products, and updating product stock.

Outcome

This practice improved my understanding of MongoDB data management, CRUD operations, query filtering, and basic real-world database usage.